Decisiones de diseño

Sistema de reservas de hotel

Silvana Arenas Molina

Kelly Stefania Martinez Martinez

Oscar Ricaurte Salamanca

Stefanny Muñoz Gasca

Lizeth Lopez Blanco

Facultad de Ciencias e Ingenierías

Universidad de Manizales

Programación I - Proyecto final

Docente

Natalia Marcela Castellanos Gomez

8 de noviembre de 2026

Decisiones de diseño del sistema de reservas de hotel

En este trabajo se presenta un sistema de reservas de hotel desarrollado en Python para el proyecto final de Programación I. El programa permite registrar clientes, consultar habitaciones, crear reservas, calcular facturas y cancelar reservas. A través de estas operaciones se aplican los conceptos de programación orientada a objetos solicitados en la actividad (Castellanos Gomez, s. f.).

Se eligió este contexto porque permite representar relaciones entre objetos con situaciones fáciles de reconocer. Un cliente realiza una reserva para una habitación y esa reserva tiene unas fechas de entrada y salida. El programa funciona por consola y guarda la información mientras está abierto. La disponibilidad depende del estado de la habitación, por lo que una reserva confirmada la mantiene ocupada hasta que se cancela.

Descripción de la solución

El sistema comienza en main.py. Al ejecutarlo se crean ocho objetos iniciales, correspondientes a un hotel, cuatro habitaciones y tres clientes. Las habitaciones incluyen los tipos sencilla, suite y familiar. Estos datos permiten consultar el programa desde el inicio, aunque también se pueden registrar clientes adicionales.

El usuario selecciona un área desde el menú. Para crear una reserva debe indicar el documento de un cliente registrado, el número de habitación, las fechas y la cantidad de huéspedes. Si los datos son válidos, el hotel confirma y guarda la reserva. Después se puede consultar su factura o cancelarla. Cada reserva crea, además, su propio objeto Estadia.

La clase Factura recibe la reserva y calcula el subtotal, el impuesto y el total. Se utiliza una tasa del 19 %, según el acuerdo del proyecto. Por ejemplo, una habitación sencilla de 100000 por noche, reservada durante dos noches, produce un subtotal de 200000, un impuesto de 38000 y un total de 238000. Para trabajar con estos importes se usa Decimal y redondeo a dos decimales.

Organización del proyecto

El código se organizó con modelos, vistas y controladores. Los modelos guardan la información y contienen las reglas del sistema. Las vistas piden datos y muestran los resultados. Los controladores reciben las entradas de la vista y llaman a los métodos necesarios. Esta separación ayuda a ubicar cada responsabilidad y facilita revisar los archivos del equipo.

Cliente guarda documento, nombre y correo. Hotel reúne las listas de clientes, habitaciones y reservas. Habitacion contiene los datos comunes del alojamiento y sus clases hijas definen los servicios particulares. Estadia valida las fechas. Reserva relaciona al cliente con la habitación y las noches de alojamiento. Factura calcula los importes asociados con esa reserva.

MenuController organiza las áreas del programa. ClienteController coordina el registro y la búsqueda de clientes. HabitacionController permite listar habitaciones, consultar detalles y comparar sus tarifas. ReservaController permite crear, consultar y cancelar reservas, además de emitir o consultar sus facturas.

Encapsulamiento y validaciones

En los modelos se utilizaron atributos privados para guardar los datos. Las propiedades permiten consultar los atributos y, cuando corresponde, modificarlos mediante setters. En Cliente, el constructor utiliza estos setters para validar desde el principio que documento, nombre y correo sean textos no vacíos y que el correo contenga arroba.

Hotel guarda sus listas como atributos privados. Cuando se consulta una lista, devuelve una copia. Así, borrar un elemento de esa copia no modifica la colección del hotel. El registro también revisa que no se repitan los documentos de los clientes ni los números de las habitaciones.

En Reserva se dejaron propiedades de consulta para sus datos. El estado cambia mediante confirmar() y cancelar(). De esta manera, el programa revisa las condiciones antes de

ocupar o liberar una habitación. Estadia también permite consultar las fechas, pero no cambiarlas directamente después de validar el período.

Clase abstracta y herencia

Habitacion se definió como una clase abstracta porque reúne información común, pero necesita que cada tipo de alojamiento indique cómo calcula su tarifa y qué servicios ofrece. Para implementarlo se usaron ABC y abstractmethod. Una clase con métodos abstractos pendientes no puede instanciarse directamente en Python (Python Software Foundation, s. f.-a).

De Habitacion heredan HabitacionSencilla, HabitacionSuite y HabitacionFamiliar. La relación tiene sentido porque cada una es una habitación. Las tres comparten número, piso, precio base, capacidad y estado. También heredan las operaciones ocupar() y liberar().

Cada hija agrega información propia. La sencilla tiene tipo de cama, la suite tiene jacuzzi y vista, y la familiar tiene camas adicionales. Así se aprovechan los datos comunes y se mantienen las diferencias necesarias para calcular cada alojamiento.

Sobreescritura y polimorfismo

Las tres clases hijas sobrescriben calcular_tarifa(), describir_servicios() y mostrar_informacion(). La sencilla multiplica el precio base por las noches. La suite suma los recargos de sus servicios. La familiar agrega el costo de sus camas adicionales. Las descripciones y la información mostrada también cambian según los atributos de cada clase.

El polimorfismo se demuestra en mostrar_habitaciones(), dentro de main.py. La función recibe una lista que contiene habitaciones de diferentes clases y la recorre con un solo ciclo. En cada vuelta llama a los mismos métodos para consultar información, servicios y tarifa. Cada objeto responde con la versión que tiene definida en su clase.

Por eso el recorrido no necesita preguntar si el objeto es una suite, una sencilla o una familiar. La comparación de tarifas del área Habitaciones aplica el mismo principio. Los condicionales de la suite se usan para calcular sus servicios adicionales.

Relaciones y composición

Hotel recibe habitaciones y clientes que ya fueron creados. Se consideró una relación de agregación porque esos objetos pueden existir por separado. Reserva mantiene referencias a un cliente y a una habitación. Factura, a su vez, mantiene una referencia a la reserva que utiliza para calcular sus importes.

La composición se aplica entre Reserva y Estadia. La reserva construye su estadía internamente a partir de las fechas recibidas. Ese objeto forma parte del período de alojamiento de la reserva. Se eligió esta relación porque una reserva tiene una estadía. La herencia no expresa esa relación.

Al crear una reserva, Hotel busca el cliente y la habitación en sus listas. Se usan los objetos registrados para evitar que otro objeto con el mismo número de habitación aporte una capacidad o un precio diferente. También se revisa que el código de reserva no esté repetido.

Excepciones del sistema

Se definieron cinco excepciones personalizadas en exceptions/hoteleras.py. Todas heredan de HotelError. Esto permite capturarlas con un tipo común y mostrar un mensaje cuando ocurre una situación del hotel. En Python las excepciones permiten tratar un error durante la ejecución y continuar cuando el programa cuenta con un manejador adecuado (Python Software Foundation, s. f.-b).

FechasInvalidasError se lanza desde Estadia si la salida es igual o anterior a la entrada. CapacidadExcedidaError se lanza desde Reserva si la cantidad de huéspedes supera la capacidad. HabitacionNoDisponibleError aparece cuando se intenta reservar o confirmar una habitación ocupada.

HabitacionNoEncontradaError y ClienteNoEncontradoError se lanzan desde las búsquedas de Hotel cuando el registro no existe. Para entradas vacías, códigos repetidos, cantidades inválidas y cambios de estado que no corresponden, se utiliza ValueError.

Las excepciones se lanzan en los modelos y se capturan en los controladores que ejecuta el programa principal. En clientes y reservas se conserva el submenú después del error. En las búsquedas de habitaciones el mensaje se muestra desde el menú principal. En ambos casos el usuario puede continuar utilizando el sistema.

Correcciones durante el desarrollo

Al unir los archivos aparecieron diferencias en los nombres utilizados. Se ajustaron los métodos de las habitaciones a snake_case y se corrigieron llamadas a propiedades que tenían paréntesis. Esas llamadas causaban TypeError al intentar mostrar la información. En Factura también se adoptó el nombre codigo para coincidir con el UML.

La primera versión de Reserva consultaba cliente.apellido y habitacion.tipo. Esos atributos no existían en los modelos acordados, por lo que se reemplazaron por sus datos reales. También se agregó el uso de las excepciones de capacidad y disponibilidad y se corrigió la confirmación para ocupar la habitación y la cancelación para liberarla.

Se revisó además la cancelación repetida. Si una reserva anterior se cancela por segunda vez, podría liberar una habitación que ya se asignó a otra reserva. Por eso cancelar() solo permite actuar sobre una reserva confirmada. Cuando una creación falla, la reserva no se agrega a la lista del hotel.

Pruebas de funcionamiento

Para verificar el código se utilizó unittest, que permite organizar pruebas y comprobar los resultados esperados (Python Software Foundation, s. f.-c). La instrucción python -m unittest discover -v ejecuta las pruebas del proyecto. En la versión revisada pasan 77 pruebas y ninguna queda omitida.

Las pruebas incluyen las clases del sistema y el recorrido desde main.py. Se comprueba que un usuario pueda registrar un cliente, crear una reserva, consultar dos veces su factura, cancelar y volver a reservar la misma habitación. También se verifican datos inválidos y la continuidad del programa después de mostrar los mensajes de error.

Uso de inteligencia artificial

Se utilizó ChatGPT de OpenAI para explicar conceptos, generar partes del código, preparar pruebas, revisar errores de integración y apoyar la redacción del documento. La generación incluyó las excepciones, Estadia, archivos de interacción de clientes, pruebas y la integración de reservas y facturación.

Silvana informó que utilizó Claude de Anthropic para comprender el UML y revisar el código y las pruebas de su sección. Su explicación de Cliente, Hotel y consultas de habitaciones se incorporó al documento y se ajustó al funcionamiento de la versión integrada.

Las propuestas se revisaron y corrigieron durante el desarrollo. Entre los cambios estuvieron la tasa de factura acordada, los nombres de métodos y atributos, las llamadas a propiedades y el estado de las habitaciones. Para verificar el resultado se ejecutaron las pruebas con las clases reales. Cada integrante debe revisar esta declaración y completar cualquier uso adicional de IA antes de entregar.

Conclusiones

El proyecto permitió aplicar los conceptos del curso en operaciones relacionadas. La herencia ayuda a compartir los datos de las habitaciones, mientras que la sobreescritura y el polimorfismo permiten utilizar tarifas y servicios diferentes dentro de un mismo recorrido. La composición representa la estadía como parte de la reserva y las excepciones permiten continuar después de un error.

Una dificultad fue mantener los mismos nombres entre el código y el diagrama. Al unir las ramas se encontraron fallos que no aparecían al trabajar por separado. Las pruebas ayudaron a revisar los modelos y el recorrido completo desde el menú.

Referencias

Castellanos Gomez, N. M. (s. f.). *Actividad colaborativa número 3 Proyecto final* [Guía de actividad]. Universidad de Manizales.

Python Software Foundation. (s. f.-a). *abc — Abstract base classes*. Documentación de Python. https://docs.python.org/es/3/library/abc.html

Python Software Foundation. (s. f.-b). *Errores y excepciones*. Documentación de Python. https://docs.python.org/es/3/tutorial/errors.html

Python Software Foundation. (s. f.-c). *unittest — Unit testing framework*. Documentación de Python. https://docs.python.org/3/library/unittest.html